Borla

Technical Debt Plan

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Technical_Debt_Plan.pdf · Version 1.0 · 13 August 2026

1. How debt was tracked during the build

Every deliberate shortcut was written down **as the decision was made**, not reconstructed afterwards — most of the items below are quoted almost verbatim from code comments left at the exact line where the trade-off was taken (e.g. `server/src/jobs/index.ts`, `server/src/modules/auth/routes.ts`). This is the practice the course asks for: technical debt as a first-class, visible decision, not a silent shortcut discovered later.

Each item below follows **Debt → Cause → Impact → Priority → Proposed Resolution**, and is labelled as one of:

- 🟢 **Acceptable temporarily** — correct trade-off for a graded demo at pilot scale; revisit only if the project continues past this exam.
- 🟡 **Scheduled for future resolution** — fine for now, but must be fixed before real users or a meaningful scale-up.
- 🔴 **Critical** — must not reach real users in its current form.

2. Debt register

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-01	~~No AI key by default~~ — **resolved**: a real `GEMINI_API_KEY` is now set (Render + local `.env` for testing), and moderation actually calls Gemini instead of falling straight to the manual queue. A related defect (D-08) surfaced the moment the key went live: the hardcoded model ID (`gemini-2.5-flash`) returned `404 – no longer available to new users` for this key's account tier, so every call silently fell back to fail-closed/manual. First fix attempt (`gemini-flash-latest` alone) still wasn't confirmed live; root-caused by testing all six plausible model IDs directly against the real key — `gemini-2.5-flash` / `gemini-2.0-flash` both 404, `gemini-3.6-flash` / `gemini-3.5-flash` / `gemini-3.7-flash` / `gemini-flash-latest` all work. Rewritten to try an ordered candidate list and cache whichever one succeeds per process, so the *next* Gemini deprecation degrades to "skip to the next candidate" instead of silently going dark again.	No Google AI Studio account was provisioned for the *original* build; the moderation pipeline (`server/src/ai/moderation.ts`) checks `GEMINI_API_KEY` and no-ops if absent — that fail-closed behaviour is exactly why D-08 degraded silently to the manual queue instead of breaking the review flow.	Confirmed end-to-end against the real key, locally: `classifyText("Great pickup, right on time, very professional!", "review")` returned `{"verdict":"allow","confidence":0.99,...}`, a real classification, not a fallback null.	🟢 Resolved	Re-run the same check once directly against production (a review, confirm it publishe without landing in the manu queue) — the code path is pr this is now just confirming t *deployed* process has the sa var.
TD-02	~~No real SMS gateway~~ — **mostly resolved**: `server/src/utils/sms.ts` sends the OTP via GiantSMS when `GIANTSMS_API_TOKEN` / `GIANTSMS_SENDER_ID` are configured, falling back to the in-app `devOtp` display only if the send fails or isn't configured. **Confirmed live in production**: `POST /api/auth/otp/request` against `https://borla.onrender.com` returned `{"delivered":true}` — GiantSMS accepted the reconstructed request shape (auth header, `from` / `to` / `msg` body) for real, not just in theory. A related gap closed at the same time: the two seeded demo numbers are arbitrary, not real handsets, so once real SMS was wired up they would have started reporting `delivered:true` while the code vanished into a gateway with no phone behind it — a graded demo would have been locked out. Fixed by special-casing `DEMO_PHONES` (`server/src/seed.ts`) to always use a fixed, documented code (`DEMO_OTP`) and never attempt a real send, regardless of gateway state.	The HTTP contract was reconstructed from published third-party client libraries, not a first-party OpenAPI spec. Production accepted the request; actual handset delivery to a *real* number has still not been visually confirmed (by definition, the two numbers this build can freely test against are the exempted demo ones).	Low residual risk: the gateway accepting the request end-to-end (not just returning a generic 200) is strong evidence the integration is correct for a genuine phone number, but "accepted by the gateway" and "arrived on a phone" are not identical.	🟡 Scheduled (down from 🔴 Critical — a real send now provably works at the HTTP layer, and grading no longer depends on it working at all)	Confirm one delivery to a re handset, then remove the d fallback field for non-demo numbers entirely so a failed surfaces as a retryable error of a security bypass.
TD-03	**Native background geolocation was not built.** Collectors only report position while their browser tab is open and in the foreground (`useGeolocation` `watchPosition`, no Capacitor/native service).	Requires an Android toolchain, a background-location plugin (commercial or complex to configure), and a physical/emulated device to validate "screen-off roaming" — none available, and a native APK isn't gradable as a "Live Application URL" per the exam's own submission format.	This is the single biggest gap from the original design's own risk register (`borla-technical-design.md` §14, risk #1) — a collector who backgrounds the app or locks their phone stops updating position, which is exactly the failure mode that erodes trust in the map.	🔴 Critical (for any real pilot) / 🟢 Acceptable for this exam	Wrap the existing shared Re screens in Capacitor, add `@capacitor-community/background-geolocation`, and test on re end Android hardware (Tecno/Infinix/itel) per the risk mitigation — no server-change needed, since the AP already accepts position upd from any authenticated clie
TD-04	~~No offline PWA shell~~ — **mostly resolved**: `vite-plugin-pwa` now ships a real installable PWA — manifest + icons (`client/public/icon-*.png`), a Workbox service worker precaching the app shell (12 entries, ~490KB), an explicit "Update available" banner (`registerType:'prompt'`, `src/pwa/UpdatePrompt.tsx` — never silently swaps content under an open tab) and a real "Install app" button	The remaining gap is genuinely more work (background sync / an offline action queue), not a missing dependency.	Households still see a blank state for anything requiring a live fetch while offline (map data, requests, broadcasts) — the shell now loads, the *data* doesn't.	🟢 Acceptable (down from 🟡 Scheduled — the install/update half is done; only the	Add a Workbox Background queue (or a hand-rolled Ind outbox) for "clear pin" and similarly small mutating act they survive a dropped conn

ID	Debt	Cause	Impact	Priority	Proposed resolution
	(`src/pwa/InstallButton.tsx` , `beforeinstallprompt`). Verified: the service worker registers and activates (checked via a headless-browser `navigator.serviceWorker.getRegistrations()` call against the built `dist/`), and API/socket traffic is deliberately excluded from precaching so it's never served stale. **Still open**: households' geolocation is on-demand/foreground-only (unchanged), and there's no queued-writes-while-offline behaviour yet (e.g. "clear pin" typed while offline doesn't queue and flush on reconnect — the shell loads offline, but mutating actions still need a live connection).			offline-writes half remains)	
TD-05	~~No Redis/BullMQ~~ — **resolved**. A real Upstash Redis instance now backs exactly what the original design specified: `presence:{id}` / `geo:collectors` / `heartbeat:zset` for live presence (`server/src/redis/presence.ts`), `pins:active` for broadcast pins, `ratelimit:otp:{phone}` / `ratelimit:notif:{id}` for rate limiting (`server/src/redis/rateLimit.ts`), a Socket.IO Redis adapter (`@socket.io/redis-adapter`) for cross-instance event delivery, and BullMQ replacing all four `node-cron` sweeps plus two new jobs — `fanout` (broadcast candidate-matching/notification, off the request thread with retries/backoff) and `moderate` (the Gemini call, same). Postgres/PostGIS stays the durable mirror, exactly per the original hot-path/cold-path split.	— (closed)	— (closed)	🟢 Resolved	Verified end-to-end against local Redis (Docker, mirrori exact Upstash `rediss://` U shape used in production): a collector going online is visil `ZCARD geo:collectors` / E `presence:{id}` ; creating a broadcast registers `pins:ac` synchronously and produces `broadcast_notifications` via the async `fanout` job w ~1s; clearing the pin remove from `pins:active` . 46 serv passing (up from 39), includ new coverage of the rate lim the async fan-out path. Prod verification pending the user pasting `REDIS_URL` into Re dashboard (`sync: false` i `render.yaml` , same patter other secrets).
TD-06	**Admin auth has no 2FA** — phone + password only (`bcrypt` -hashed), same JWT mechanism as household/collector, instead of the original design's "stronger auth than the field apps."	Full 2FA (TOTP/SMS) needs either the same missing SMS gateway (TD-02) or a new TOTP flow — both extra scope for an account class that, in this build, is a single seeded demo user.	An admin account compromise is higher blast radius than a household/collector one (can suspend users, remove content) — acceptable only because there is exactly one demo admin account behind a strong generated password, never exposed publicly except in the graded credentials file.	🔴 Critical (before real ops staff use it)	Add TOTP (e.g. `otplib`) ga behind the existing `role='a` check; store a `totp_secret` column already anticipated l schema's extensibility (JSON `app_config` pattern could or a dedicated column).
TD-07	The standalone `admins` table from the original design was **folded into** **`users.role = 'admin'`** with a nullable `password_hash` column that only admins use.	Reduces schema surface for a single-admin demo; avoids a parallel user model.	Slight modelling smell — `password_hash` is `NULL` for 99% of rows (households/collectors) — and there is no dedicated `admins.role` tier (`ops` / `moderator` / `superadmin` from the original design).	🟢 Acceptable	If multiple admin tiers are e needed, split back into a ded `admins` table with its own r enum, migrated via a straightforward `INSERT ..` `SELECT` from `users WHERE` `role='admin'` .
TD-08	**Leaflet + raw OpenStreetMap raster tiles** instead of the original design's MapLibre GL + vector tiles.	Zero-config: no tile-provider account/key needed, which matters when nobody is provisioning new accounts mid-exam.	Slightly less crisp rendering and no offline vector caching; functionally equivalent for markers/popups/radius visualisation, which is all this build needs.	🟢 Acceptable	Swap the `MapView` compon tile layer for a MapLibre + v source once a tile provider ((MapTiler free tier) is chosen component's public interfac (`center` , `points`) would r need to change.
TD-09	**No i18n beyond English**, despite the original design and the low-literacy design brief calling for Twi/Ga from day one.	Time-boxed; English-only was the fastest path to a fully functional, testable app across every screen.	Directly works against the stated accessibility goal for the target users.	🟡 Scheduled	The UI already isolates copy JSX rather than scattering it introduce `react-i18next` originally-planned `packages/shared/i18n`) a extract strings; start with Tv Ga per the pilot-neighbourh plan.
TD-10	**Automated test coverage is broad, not deep.** 46 server tests cover every route's happy path and its most important failure mode (idempotency guards, role gates, moderation gating, phone normalization, Redis rate limiting, async job side-effects); there is no fuzzing, no load testing, and client tests cover 2 of ~9 components.	A time-boxed exam favoured "every module has real integration tests" over "one module is exhaustively tested" — a deliberate breadth-over-depth call, consistent with §7.7 of the SRS.	Edge cases in less-tested paths (e.g. concurrent double-accept races under real network latency, malformed geo payloads) are not proven safe, only reasoned about.	🟡 Scheduled	Add property-based tests for geospatial radius math, a concurrency test that fires accept+reject simultaneousl against the same request to p the conditional-update guar real contention (not just seq calls), and React Testing Lib coverage for `HouseholdHome` / `Collecto`
TD-11	**Photo → waste-type AI classification was not built** (§18 Job B in the original design).	Explicitly deprioritised behind moderation (Job A) per the original design's own value ranking — moderation is the safety-critical	Households still pick a waste-type tile manually instead of snapping a photo — a real but non-blocking gap for low-literacy accessibility.	🟡 Scheduled	Reuse the existing `aiClien` shaped moderation module' pattern: a second thin wrapp calling Gemini's multimodal

ID	Debt	Cause	Impact	Priority	Proposed resolution
		one; photo classification is a UX nicety.			endpoint, wired to a new `PC` `/broadcasts` image field, cl side downscaled before uplo
TD-12	**Provider call-masking was not built** — accepted requests reveal each party's real phone number ("reveal-on-accept", the original design's own recommended v1 approach, §10 option 1).	This *is* the design's own recommended v1 trade-off, not a shortcut taken under exam pressure — kept as-is deliberately.	Real phone numbers are exchanged between two people who chose each other via an accepted request — acceptable given the low stakes described in the original design.	🟢 Acceptable	Add provider number-mask (e.g. via Hubtel) only if abus reports or scale warrant the per-call cost, exactly as the c design already prescribes.
TD-13	**Single Render Web Service, no staging environment, no CI pipeline beyond local `npm test`.**	Free-tier, single-service deploy was the fastest path to a real "Live Application URL" within the exam window.	A bad commit reaches production directly; no automated gate before deploy.	🟡 Scheduled	Add a GitHub Actions workf running `npm test` on every (config-only change, no app change needed) and a secon Render environment for sta before promoting to the grad URL.
TD-14	**The collector↔household route (once a direct request is accepted) calls OSRM's public demo routing server** (`router.project-osrm.org`) directly from the browser — no API key, but also no SLA, rate-limit guarantee, or uptime commitment.	Zero-config, matching the same trade-off already made for the OSM raster tile layer (TD-08): a routing feature needed to exist for this build, and every account-gated alternative (Mapbox Directions, Google Directions) needed a key nobody had time to provision.	If the demo server is down, slow, or rate-limits this app's traffic, the route silently degrades to a straight line between the two points (`client/src/components/MapView.tsx`, `useRoute`'s fallback) rather than failing — correct fail-safe behaviour, but the *quality* of the route (does it actually follow real roads) is not guaranteed at any given moment.	🟢 Acceptable	If usage grows, move to a se hosted OSRM instance or a Directions API with an SLA `useRoute` hook's interface (`from` / `to` in, a lat/lon pat distance/duration out) woul need to change, only the fetc inside it.

3. Debt NOT taken — deliberate design choices worth naming

To be explicit about the difference between *debt* (a shortcut that should eventually be paid down) and a *considered trade-off that stays*: the no-collector-binding broadcast plane, the absence of in-app payments, and reveal-on-accept-instead-of-provider-masking are not technical debt — they are the original design's own architecture, kept unchanged because they are still the right call at this scale (see `borla-technical-design.md` §1 and §10, and TD-12 above).

4. Repayment plan by phase

This maps directly onto `Project_Documentation.md` §15 (Maintenance & Future Evolution):

Phase	Items paid down
v1.1 (immediate post-submission, if continued)	TD-06 (admin 2FA) — the one remaining 🔴 Critical item, must land before any real admin touches the system; confirm one real GiantSMS handset delivery to fully close TD-02
v1.2	TD-13 (CI pipeline)
v1.3	TD-03 (native Capacitor collector app + background geolocation) — the single biggest lift, matched to the original design's own phased roadmap
v2.0	TD-09 (i18n), TD-04 (offline write queue — the install/update half already shipped), TD-11 (photo classification)
Ongoing	TD-10 (expand test depth every time a new bug class is found in production)